

Teacher-Anchored and Statistical Calibration for Drift-Adaptive Affine GKP Decoding

Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) quantum error correction encodes a qubit into an oscillator and uses continuous-valued modular syndromes to infer displacement errors. In realistic finite-energy GKP states, syndrome statistics are shaped by state-preparation width, measurement noise, circuit-level imperfections, and slow calibration drift. This work studies a two-timescale decoding architecture for such settings. A low-latency fast loop applies an affine correction $\Delta_t = K_t s_t + b_t$, while a slower calibration loop estimates the current effective noise state from recent syndrome histograms and updates the runtime parameters. This keeps the per-shot path compatible with fixed-point hardware execution while allowing the decoder to adapt to nonstationary noise.

The architecture is designed around a narrow and deployment-oriented role for learning. A classical teacher estimator provides an interpretable baseline for the affine parameters, and a lightweight CNN predicts only a bounded residual calibration term, currently focused on the bias b_t . This differs from end-to-end neural decoders and from outer-code soft-information decoders: the learned module updates a low-dimensional physical-layer calibration surface, whereas the real-time path remains a deterministic affine rule. The intended systems advantage is to reduce logical failures under drift while keeping the per-shot latency, resource footprint, and verification burden close to those of a small fixed-point affine controller. Under a locked four-scenario software-HIL protocol, adaptive affine calibration with a teacher-anchored residual branch improves over filtering baselines in all four frozen scenarios. Bounded multi-seed and ablation materials provide descriptive support for this reading and show that histogram-delta features materially affect the learned branch, while teacher-side inputs do not admit a simple necessity claim. A separately labeled statistical-calibration extension lane further suggests that simple non-neural estimators can be strong under the same affine runtime contract, but its accepted closure remains supplement-side and no-promotion because no unique clean threshold has been established.

Contents

1	Introduction	2
2	Summary of Contributions	3
2.1	Metric-level advantages	4
3	Brief Review of the GKP Code	4
3.1	Ideal and approximate code states	4
3.2	Syndrome measurement as modular displacement information	5
3.3	Local affine decoding and its limits	5
3.4	Logical failure criterion	6

4	Noise and Drift Model	6
4.1	Effective noise state	6
4.2	Noise sources represented by the model	6
4.3	Drift scenarios	6
5	Model Architecture	7
5.1	Fast loop: affine fixed-point decoder	7
5.2	Parameter mapping	7
5.3	Teacher estimators	7
5.4	CNN residual branch	8
5.5	Statistical calibration branch	8
5.6	Stage-and-commit runtime contract	8
6	Relationship to Existing Work	8
6.1	GKP and bosonic analog soft information	8
6.2	Adaptive priors and syndrome-statistics estimation	9
6.3	Learned low-latency QEC modules	9
6.4	Real-time and FPGA QEC decoders	9
6.5	Advantages relative to existing QEC decoder approaches	9
7	Experimental Setup	10
7.1	Software-HIL protocol	10
7.2	Scenarios and modes	10
8	Numerical Results and Benchmark Plan	11
8.1	Four-scenario affine benchmark	11
8.2	Feature and teacher ablations	11
8.3	Statistical calibration as a bounded extension lane	11
8.4	Local sensitivity inside the statistical-calibration extension lane	11
8.5	Per-scenario best variants and the no-promotion gate	11
8.6	Mechanism probe for residual-b behavior	12
8.7	Unseen drift generalization	12
8.8	Oracle and wrapped-Gaussian baselines	12
8.9	Runtime, quantization, and fixed-point degradation	12
8.10	Embedded runtime and board-level validation	12
8.11	Parallel sidecar extension waves	12
9	Discussion	14
10	Conclusion	15

1 Introduction

Bosonic codes protect quantum information by encoding a logical qubit into the Hilbert space of an oscillator. The GKP code is a central example: ideal code states form a periodic lattice in phase space, so small shift errors can be diagnosed through modular syndrome measurements and corrected by displacement operations [1, 2]. Unlike binary stabilizer syndromes, GKP syndromes

are analog. Their continuous values carry information not only about the likely displacement, but also about how close the correction is to a lattice decision boundary.

The practical challenge is that physical GKP states are approximate finite-energy states. Their comb peaks have finite width and an envelope, and syndrome measurements are further affected by measurement inefficiency, noisy auxiliary states, circuit-level imperfections, oscillator loss, and calibration error [2, 9]. Consequently, the effective noise model seen by the decoder can change over time. A fixed syndrome-to-correction map may be well tuned for one operating point but mismatched after bias, variance, or covariance drift.

Recent GKP and bosonic-decoding work makes clear that analog soft information is valuable. Surface-GKP and concatenated bosonic-QLDPC decoders use continuous syndrome information to set matching weights, belief-propagation messages, or effective outer-code priors [4, 5, 6, 7, 8]. At the same time, modern QEC decoder systems show that learned modules are most compelling when their latency role and integration boundary are explicit [15, 16]. Hardware and calibration-oriented studies similarly emphasize that decoder priors must be kept aligned with device statistics [10, 11, 12, 13, 14].

This work applies that systems principle at the physical GKP correction layer. The fast loop is intentionally simple:

$$\Delta_t = K_t s_t + b_t, \tag{1}$$

where s_t is the measured two-quadrature GKP syndrome and (K_t, b_t) are quantized runtime parameters. The slow loop observes recent syndrome histograms, estimates an effective noise state, and updates (K_t, b_t) at a lower rate. The design therefore separates two concerns: the per-shot correction path is a bounded affine operation, while the nonstationary estimation problem is handled asynchronously through calibrated parameter updates.

The main advantage of this formulation is its combination of physical structure, adaptation, and deployment discipline. Compared with a fixed affine decoder, the parameters can track drift. Compared with a full neural decoder, the online computation is smaller and more predictable. Compared with outer-code soft-information methods, the adaptation target is a physical-layer GKP displacement rule rather than a matching graph or LDPC message schedule. Compared with calibration-conditioned neural decoders, the slow context does not enlarge the per-shot path; it only changes the affine parameters staged into the runtime bank.

This also gives the work a specific hardware-facing story. Much of the real-time FPGA-QEC literature focuses on decoding binary stabilizer syndromes for surface codes, repetition-code experiments, or quantum LDPC memories using lookup tables, union-find variants, matching or greedy approximations, clustering, neural decoders, or message-passing hardware [18, 19, 20, 21, 22, 24, 25, 27]. Those results establish the standard for closed-loop latency and resource reporting, but they do not directly address a GKP physical correction layer, where the measured syndrome is analog, modular, and shaped by finite-energy state preparation. The present architecture is therefore complementary to those decoders: it provides a low-dimensional analog calibration layer that could sit before, inside, or alongside a larger stabilizer or LDPC decoding stack while keeping the per-shot action as a small fixed-point affine displacement.

2 Summary of Contributions

The work can be organized around five contributions.

1. A two-timescale adaptive affine formulation for approximate GKP decoding. The per-shot correction path is formulated as a bounded affine estimator $\Delta_t = K_t s_t + b_t$. The expensive

adaptation problem is moved into a slow loop that estimates the current effective noise state from recent syndrome statistics. This links the continuous-syndrome GKP picture to a runtime contract that can be implemented as a quantized matrix-vector operation.

2. A teacher-anchored residual calibration strategy. A classical teacher estimator produces a stable baseline $(K_t^{\text{teacher}}, b_t^{\text{teacher}})$. The CNN predicts a small residual,

$$\delta b_t = f_\phi(H_{t-c+1:t}, \Delta H_t, z_t^{\text{teacher}}), \quad (2)$$

and the committed parameters are

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (3)$$

Learning is therefore used to calibrate a low-dimensional control surface, not to replace the GKP decoder end-to-end.

3. A deployment-aware runtime architecture. The architecture exposes fixed-point quantization, clipping, saturation diagnostics, parameter smoothing, stale-parameter behavior, and double-buffered stage-and-commit updates. These constraints become measurable objects in the benchmark rather than assumptions added after the algorithm is designed.

4. A benchmark suite for adaptive affine calibration. The mainline numerical study compares filtering baselines and learned residual variants under four frozen drift scenarios. Bounded ablation and multi-seed follow-ups then support a conservative reading of the gains. A separately labeled statistical calibration extension lane asks whether a simple non-neural rule can also be strong inside the same runtime contract, but it remains a supplement-side no-promotion result rather than a promoted mainline comparator.

5. A bridge between GKP analog calibration and real-time decoder hardware practice. The formulation exposes a clear interface between continuous GKP syndrome statistics and a hardware-style decoder contract: the slow side may use histograms, teachers, learned residuals, or statistical estimators, but the fast side only consumes bounded quantized parameters. This makes the proposed method more directly comparable with FPGA-QEC systems than an unconstrained offline neural decoder, while targeting a GKP-specific problem that binary stabilizer decoders do not solve on their own.

2.1 Metric-level advantages

For reviewer-facing comparison, the advantages should be stated by metric rather than by architecture alone. Table 1 separates the evidence currently supported by software-HIL experiments from the hardware-facing advantages that follow from the fast-path contract but still require board-level measurement.

3 Brief Review of the GKP Code

3.1 Ideal and approximate code states

The square-lattice GKP code encodes a qubit into an oscillator by using periodic structure in the q and p quadratures [1]. In the convention used here, the lattice constant is

$$\lambda = \sqrt{2\pi}. \quad (4)$$

An ideal logical state may be represented heuristically as an infinite comb, for example

$$|\bar{0}\rangle \propto \sum_{n \in \mathbb{Z}} |n\lambda\rangle_q. \quad (5)$$

Ideal comb states are not physical because they require infinite energy. Approximate GKP states replace the infinitely sharp comb with finitely squeezed peaks and an envelope. Finite-energy structure is central to practical GKP QEC rather than a minor implementation detail [2]: it determines the intrinsic syndrome uncertainty and contributes an effective displacement-like noise.

3.2 Syndrome measurement as modular displacement information

Let the accumulated phase-space displacement error before correction be

$$e_t = \begin{bmatrix} e_{q,t} \\ e_{p,t} \end{bmatrix}. \quad (6)$$

An ideal GKP syndrome observes this displacement modulo the lattice:

$$s_t = e_t \bmod \lambda, \quad s_{q,t}, s_{p,t} \in [-\lambda/2, \lambda/2). \quad (7)$$

In a finite-energy and noisy-measurement setting, the measured syndrome is more accurately described as

$$\tilde{s}_t = \text{mod}(e_t, \lambda) + \eta_t^{\text{meas}} + \eta_t^{\text{GKP}}. \quad (8)$$

Here η_t^{meas} represents measurement and auxiliary-state contributions, while η_t^{GKP} summarizes finite-squeezing uncertainty. The decoder therefore observes a noisy modular representative, not the absolute displacement.

3.3 Local affine decoding and its limits

The modulo structure makes exact GKP decoding nonlinear and branch dependent. Near a single lattice branch, however, a Gaussian approximation gives a useful local model. If e and s are jointly Gaussian,

$$\begin{bmatrix} e \\ s \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_e \\ \mu_s \end{bmatrix}, \begin{bmatrix} \Sigma_{ee} & \Sigma_{es} \\ \Sigma_{se} & \Sigma_{ss} \end{bmatrix} \right), \quad (9)$$

then the linear-MMSE estimate is

$$\hat{e} = \mu_e + \Sigma_{es} \Sigma_{ss}^{-1} (s - \mu_s) = Ks + b, \quad (10)$$

where

$$K = \Sigma_{es} \Sigma_{ss}^{-1}, \quad b = \mu_e - K\mu_s. \quad (11)$$

This derivation explains why an affine fast path is a plausible low-latency approximation. It also marks the boundary of the approximation. Near lattice decision boundaries, the posterior can be multimodal; maximum-likelihood, closest-lattice-point, or wrapped-Gaussian decoders can outperform a single global affine rule [3, 9].

3.4 Logical failure criterion

After correction, the residual displacement is wrapped into the GKP fundamental cell. A logical error occurs when the residual crosses a logical decision boundary, for example

$$|r_{q,t}| > \lambda/2 \quad \Rightarrow \quad X_L \text{ error}, \quad (12)$$

and similarly for the p -quadrature. Closed-loop logical error probability is therefore the central metric; parameter-regression error is useful only when it correlates with the downstream correction outcome.

4 Noise and Drift Model

4.1 Effective noise state

The slow loop does not attempt to infer every microscopic hardware parameter. It uses a low-dimensional effective state,

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t), \quad (13)$$

where σ_t controls the displacement scale, $\mu_{q,t}$ and $\mu_{p,t}$ represent mean bias, and ϑ_t captures covariance-axis rotation. This compact parameterization can be mapped to affine runtime parameters and estimated from finite histogram windows.

4.2 Noise sources represented by the model

The model is an effective displacement-noise model with additional measurement uncertainty. It absorbs several physical effects:

1. finite-energy GKP peak width and envelope effects;
2. Gaussian random displacement noise in q and p ;
3. biased displacement means caused by calibration offsets;
4. anisotropic or rotated covariance caused by asymmetric noise;
5. syndrome measurement noise and noisy auxiliary-state contributions.

This is narrower than a full circuit-level GKP noise model, but it is aligned with the affine fast-path contract.

4.3 Drift scenarios

The benchmark suite uses four drift scenarios:

1. `static_bias_theta`: a biased and rotated operating point;
2. `linear_ramp`: slowly changing effective noise parameters;
3. `step_sigma_theta`: a sudden variance/orientation change;
4. `periodic_drift`: periodic nonstationarity.

These scenarios separate different adaptation demands: steady calibration, tracking, shock response, and periodic following.

5 Model Architecture

5.1 Fast loop: affine fixed-point decoder

The fast loop receives the current measured syndrome s_t , reads the active parameter bank, and computes

$$s_t^{\text{clip}} = \text{clip}(s_t, -s_{\max}, s_{\max}), \quad (14)$$

$$\Delta_t^{\text{raw}} = K_t s_t^{\text{clip}} + b_t, \quad (15)$$

$$\Delta_t = Q(\text{clip}(\Delta_t^{\text{raw}}, -\Delta_{\max}, \Delta_{\max})), \quad (16)$$

where $Q(\cdot)$ denotes fixed-point quantization. The implementation uses an FPGA-oriented Q4.20 representation for syndrome values and runtime parameters. The fast loop also records diagnostic counters, including histogram-input saturation, correction saturation, overflow, aggressive parameter events, commit counts, and fallback-related signals.

5.2 Parameter mapping

Given an estimated noise state, the parameter mapper constructs an error covariance

$$C = R(\vartheta) \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_p^2 \end{bmatrix} R(\vartheta)^\top, \quad (17)$$

and a measurement covariance

$$R_{\text{meas}} = (\sigma_{\text{meas}}^2 + \Delta_{\text{eff}}^2)I. \quad (18)$$

The raw gain is

$$K_{\text{raw}} = C(C + R_{\text{meas}})^{-1}, \quad (19)$$

followed by gain clipping or scaling. The bias target is

$$b_{\text{target}} = \alpha(I - K_{\text{target}})\mu, \quad \mu = \begin{bmatrix} \mu_q \\ \mu_p \end{bmatrix}. \quad (20)$$

Finally, exponential smoothing produces staged parameters:

$$K_t = (1 - \beta)K_{t-1} + \beta K_{\text{target}}, \quad b_t = (1 - \beta)b_{t-1} + \beta b_{\text{target}}. \quad (21)$$

5.3 Teacher estimators

The teacher family provides stable estimates of θ_t^{noise} from recent syndrome history. A simple teacher computes moments over a histogram window. Stronger teachers such as EKF, UKF, RLS, or particle-filter variants add state-space assumptions over time. Abstractly,

$$\hat{\theta}_t^{\text{teacher}} = \text{Teacher}(H_{1:t}), \quad (22)$$

and

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \quad (23)$$

The teacher keeps the learned branch near an interpretable calibration surface and creates meaningful ablation baselines.

5.4 CNN residual branch

The CNN consumes a short context of normalized syndrome histograms and histogram deltas:

$$X_t^{\text{hist}} = [H_{t-c+1}, \dots, H_t, \Delta H_{t-c+2}, \dots, \Delta H_t]. \quad (24)$$

Teacher-side features may include

$$z_t^{\text{teacher}} = (\hat{\theta}_t^{\text{teacher}}, K_t^{\text{teacher}}, b_t^{\text{teacher}}, \Delta b_t^{\text{teacher}}, \dots). \quad (25)$$

The gated branch narrows this to a small set of teacher- b and teacher- Δb scalars. The learned branch predicts

$$\hat{\delta b}_t = f_\phi(X_t^{\text{hist}}, z_t^{\text{teacher}}), \quad (26)$$

then clips the result:

$$\delta b_t = \text{clip}(s_b \hat{\delta b}_t, -b_{\max}, b_{\max}). \quad (27)$$

The committed parameter is

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (28)$$

5.5 Statistical calibration branch

The same affine runtime contract also supports a non-neural statistical calibration branch. This branch estimates a bounded bias correction directly from recent syndrome statistics and emits runtime parameters through the same clipping, smoothing, and stage-and-commit path. Its role is both practical and scientific: it tests whether the benefit comes from a general histogram-driven calibration principle rather than from the CNN architecture alone.

5.6 Stage-and-commit runtime contract

The slow loop does not directly mutate active fast-loop parameters. It stages a candidate (K, b) into an inactive bank and commits it at a safe epoch boundary:

$$(K_t, b_t) = \begin{cases} (K^A, b^A), & t < t_{\text{commit}}, \\ (K^B, b^B), & t \geq t_{\text{commit}}. \end{cases} \quad (29)$$

This contract allows stale-parameter effects, update latency, commit success, rollback/fallback behavior, and fixed-point stability to be measured separately from logical performance.

6 Relationship to Existing Work

6.1 GKP and bosonic analog soft information

GKP analog syndrome information has already been shown to be valuable. Work on analog QEC, surface-GKP codes, and concatenated bosonic-QLDPC decoding uses continuous measurement information to set decoder weights or soft messages [3, 4, 5, 6, 7, 8]. The distinction here is that the analog statistics are compressed into physical-layer affine GKP fast-path parameters, rather than into outer-code matching weights or LDPC messages.

6.2 Adaptive priors and syndrome-statistics estimation

Adaptive-weight and syndrome-statistics methods estimate decoder weights or noise parameters from measured data [10, 11, 12]. Calibrated decoders and decoder-prior optimization show that prior mismatch can materially affect logical performance [13, 14]. This work borrows that lesson, but maps the adaptation target to the GKP affine parameters (K, b) and constrains the update through a stage-and-commit runtime contract.

6.3 Learned low-latency QEC modules

AI pre-decoders, high-accuracy neural decoders, and calibration-conditioned FiLM decoders demonstrate a systems-oriented pattern: a learned module is most useful when its input/output contract, latency role, and integration with a classical decoder are explicit [15, 16, 17]. The learned module here is not the per-shot decoder. It is a slow calibration component that updates low-dimensional affine parameters; the per-shot path remains fixed-point affine logic.

6.4 Real-time and FPGA QEC decoders

Real-time QEC decoders and FPGA-integrated systems have reported low-latency hardware decoding in stabilizer-code settings [24, 25, 26, 27]. Hardware-oriented decoder designs also include lookup-table approaches, union-find architectures, matching and greedy approximations, clustering decoders, and FPGA-tailored quantum-LDPC message-passing systems [18, 19, 20, 21, 22, 23]. Real-time bosonic QEC has also been demonstrated in experimental feedback systems [28, 29]. These works set the standard for full deployment claims: closed-loop timing, worst-case latency, resource use, and hardware integration.

The contribution here is intentionally narrower and complementary. It does not claim to replace surface-code, repetition-code, or qLDPC hardware decoders. Instead, it targets the GKP physical-layer correction problem that precedes many concatenated-code decoding tasks: how should a device update the analog displacement rule when the finite-energy GKP syndrome distribution drifts? In this setting, a fixed binary graph decoder is not the natural first abstraction, and an unconstrained neural network on the per-shot path is not an attractive hardware endpoint. A staged affine parameter bank provides a smaller contract: update (K, b) slowly from recent analog statistics, then execute only $Ks + b$ at the fast boundary.

6.5 Advantages relative to existing QEC decoder approaches

The proposed architecture is best viewed as a middle layer between physical GKP syndrome acquisition and higher-level QEC decoding. Its advantages over nearby approaches are architectural rather than claims of completed board-level deployment.

Preserving analog GKP information at the right layer. Outer-code GKP methods already show that analog syndrome values improve matching weights or belief-propagation messages. This work uses the same principle earlier in the stack: the analog histogram updates the physical GKP displacement rule itself. This can reduce prior mismatch before the information is handed to a surface-code, LDPC, or other outer decoder.

Drift adaptation without a neural real-time critical path. End-to-end neural decoders can be expressive, but they make worst-case latency, fixed-point validation, and fallback behavior harder to reason about. Here, learning is optional and narrow: a CNN residual branch or a

statistical calibration branch updates a small parameter surface, while the real-time path remains deterministic affine logic. This separation is expected to be useful when future hardware systems need both adaptation and bounded per-shot timing.

A replaceable calibration module rather than a single decoder commitment. The same runtime contract can host a teacher estimator, a learned residual branch, or a non-neural statistical calibration rule. The strong statistical calibration results therefore strengthen rather than weaken the design story: they indicate that the important object is the histogram-driven affine calibration contract, not one particular CNN architecture.

Explicit deployment diagnostics. The framework makes quantization, clipping, saturation, stale-parameter use, commit counts, fallback behavior, and update latency visible benchmark objects. This is closer to FPGA-QEC practice than reporting only offline regression error, and it provides a path to future resource and timing measurements once a real hardware host is available.

Compatibility with larger QEC stacks. Because the fast path emits a physical displacement correction rather than a complete outer-code decision, the method can in principle be combined with surface-GKP, bosonic-QLDPC, or conventional stabilizer-code decoders. Its expected role is to keep the GKP layer calibrated under drift so that downstream decoders receive better-conditioned residual information.

7 Experimental Setup

7.1 Software-HIL protocol

The numerical experiments use a software-HIL protocol that preserves the fast loop, slow loop, parameter-bank, and staged-commit interfaces while executing the underlying experiment in software. This protocol is sufficient for comparing adaptive calibration laws under identical drift trajectories and paired seeds. The locked frozen protocol is the authoritative mainline anchor for the current paper-facing result layer; later ablation, multi-seed, and extension-lane materials are interpreted relative to this anchor rather than as a rewritten benchmark. It does not measure board-level latency or hardware resource use.

7.2 Scenarios and modes

The four drift scenarios are:

Scenario key	Interpretation
<code>static_bias_theta</code>	biased and rotated operating point
<code>linear_ramp</code>	slowly changing effective noise parameters
<code>step_sigma_theta</code>	sudden variance/orientation change
<code>periodic_drift</code>	periodic nonstationarity

The mainline comparison includes filtering baselines, residual affine baselines, and the learned hybrid residual branch. Additional bounded materials then cover feature/teacher ablations, multi-seed descriptive snapshots, and a separately labeled statistical-calibration extension lane under the same runtime contract. The reported metric is `final_ler_mean`, where lower values indicate fewer logical failures under the fixed software-HIL protocol.

8 Numerical Results and Benchmark Plan

8.1 Four-scenario affine benchmark

Table 2 records the locked frozen benchmark that also underlies the paper-facing main-text visual summary. Under this software-HIL protocol, the learned hybrid residual branch is the winner in all four scenarios, with UKF as the runner-up throughout. This is the authoritative mainline result layer of the current note; it should not be retold as an expanded benchmark, a `.tflite` ranking, or a board-level ranking.

8.2 Feature and teacher ablations

Table 3 is best read as bounded appendix-side support for the frozen benchmark, not as a replacement ranking table. Removing histogram deltas degrades performance relative to the full hybrid model, indicating that temporal changes in the syndrome histogram are useful within this fixed benchmark set. Removing teacher-side channels also changes performance materially, but the no-teacher-params row performing best means the current evidence does not support a simple teacher-necessity claim. The safest reading is therefore descriptive: feature and teacher design matter, yet the bounded ablation does not close the mechanism question.

8.3 Statistical calibration as a bounded extension lane

A separately labeled extension lane adds statistical calibration variants under the same affine runtime contract. Table 4 records the bounded comparison snapshot that motivated this lane: within that task-local matrix, statistical calibration variants outperform UKF and the learned hybrid branch. However, the accepted closure deliberately does not promote this lane into the mainline comparator set. The locked five-mode frozen benchmark remains the authoritative mainline result, and the statistical-calibration lane is retained as supplement-side evidence for the broader adaptive-affine contract.

8.4 Local sensitivity inside the statistical-calibration extension lane

The local grid around the default setting shows that the extension-lane signal is not tied to a single pathological point, but it also does not identify a unique clean threshold choice. The high-threshold variant has the best aggregate mean LER by a negligible margin, whereas the default variant wins three of four scenarios and has the best mean within-lane rank. This persistent tie is why the accepted closure remains no-promotion rather than collapsing to one promoted statistical-calibration setting.

8.5 Per-scenario best variants and the no-promotion gate

Table 6 reports the per-scenario winners inside the extension lane. These rows show why the lane remains scientifically interesting, but they are not sufficient to override the no-promotion gate: the default and high-threshold settings remain persistently tied across aggregate and scenario-wise summaries, and the `static.bias.theta` winner also carries a mixed-status caveat. The safest use of these numbers is therefore supplement-side documentation of the lane, not a claim that a mature promoted comparator has been established.

8.6 Mechanism probe for residual-b behavior

A paper-facing mechanism figure is available for the learned residual branch, with appendix tables retaining the authoritative numeric snapshots. The descriptive reading is that the seed-wise instability pattern appears across more than one seed, while lowering the residual clip is mixed and often harmful. This supports a cautious interpretation: residual amplitude participates in some failure modes, but it is not a monotonic causal explanation by itself, and the current evidence does not prove teacher necessity. Appendix tables provide the bounded ablation and cross-seed numeric support for this conservative reading.

8.7 Unseen drift generalization

The current four scenarios cover steady bias, ramp tracking, step response, and periodic drift. A stronger benchmark requires unseen drift families such as random-walk drift, $1/f$ -like drift, burst measurement noise, coupled bias-variance drift, and faster-than-window drift. This experiment is needed to distinguish genuine drift adaptation from overfitting to a small set of hand-designed trajectories.

8.8 Oracle and wrapped-Gaussian baselines

The affine methods need comparison against stronger theoretical baselines: nearest-lattice hard decoding, static affine decoding, oracle affine decoding with true noise state, and wrapped-Gaussian or maximum-likelihood decoding in small settings. These baselines quantify how much performance is lost because the fast path is affine, and how much is lost because the slow loop estimates the noise state imperfectly.

8.9 Runtime, quantization, and fixed-point degradation

The deployment story depends on more than logical performance. The current paper-facing evidence therefore keeps runtime material at the boundary layer: selected preserved `.tflite` artifacts have been executed in an isolated current-host environment, but the broader fixed-point/runtime measurements listed here remain future benchmark items. Saturation and overflow rates, commit latency, stale-parameter penalty, fallback frequency, and per-update slow-loop cost are still needed to connect the mathematical affine rule to a fully measured real-time decoder contract.

8.10 Embedded runtime and board-level validation

Embedded inference and board-level validation remain separate from software-HIL simulation. Current host evidence stops at two conservative facts: selected preserved `.tflite` artifacts run in an isolated current-host environment, and the read-only real-board gate/regeneration/provenance lane still ends in `NO_GO_REAL_BOARD_HOST_OR_DEVICE_PATH_UNAVAILABLE`. Exported-model equivalence, embedded runtime latency, host-to-device update cost, hardware resource use, and closed-loop timing therefore remain future evidence rather than completed results.

8.11 Parallel sidecar extension waves

The main benchmark and paper-material lane should remain separated from parallel extension experiments. A useful way to organize the next experiments is to treat them as sidecar waves: each wave may run in parallel with the mainline, but its outputs remain sidecar evidence until a later

promotion gate decides whether a result deserves a new mainline task. In particular, these experiments must not rewrite the frozen four-scenario anchor, promote the statistical-calibration lane into a mature comparator, or turn software-HIL evidence into `.tflite` or board-level validation.

Wave A contains the immediately bounded sidecar lanes. These lanes preserve the fast-loop contract and mostly operate through cached replay, toy simulation, or deterministic contract tests.

Wave A lane	Experimental question	First safe execution
Temporal histogram TCN	Does a short causal sequence of histogram windows, deltas, and compact statistics improve residual- b or drift-regime prediction beyond the current short-window features?	Cached replay or tiny TCN residual-head probe under a bounded sidecar run root.
Adaptive teacher confidence	Can a syndrome-only or teacher-output replay estimate confidence, freeze unsafe updates, or fall back to a conservative anchor before a bad parameter commit?	Offline teacher-policy replay with explicit confidence and fallback fields.
Gain-scheduled parameter bank	Can a finite piecewise-affine bank reduce parameter jitter or improve drift tracking while keeping the fast path as $\Delta = K_i s + b_i$?	Bank-selection toy replay with range, thrashing, and switch-frequency diagnostics.
Atomic commit and rollback	Which stage, commit, acknowledgement, freeze, rollback, and timeout states are needed before stronger runtime integration is credible?	Deterministic mock contract tests only; no MMIO, DMA, or board writes.

Wave B is the second-wave candidate pool. These routes are useful for making the paper experimentally complete, but they should start as S0 design or toy probes rather than as long formal benchmarks.

Wave B lane	Experimental question	Boundary
Moments and EWMA feature probes	Do low-cost summaries such as marginals, mean/variance drift, EWMA, entropy, anisotropy, or edge-mass features explain the same gains as the full histogram image?	Feature ablation only; not a new formal benchmark by itself.
Teacher-conditioned residual head	Does conditioning the residual head on teacher state, teacher deltas, or confidence improve stability without making the learned module the per-shot decoder?	Small retrain or replay; still a bounded slow-loop calibration module.
State-space or lightweight sequence model	Are modern compressed temporal models such as tiny state-space, Mamba-like, or recurrent/transformer toy heads useful after the TCN baseline is known?	Research-first toy route; no full decoder replacement and no fast-loop change.
Outer-code soft-information positioning	How would the physical GKP calibration layer interface with surface-GKP, GKP-surface, or QLDPC-GKP soft-information decoders?	Positioning memo or tiny interface toy; not a current mainline code-family benchmark.

The purpose of this wave structure is not to multiply unrelated experiments. It keeps the scientific question fixed: the fast path remains a deterministic affine correction, while sidecar lanes ask which slow-loop estimator, feature summary, teacher policy, or commit policy most safely

updates the affine parameters under drift. Negative or inconclusive sidecar results are also useful, because they can show that the current histogram-conditioned affine contract is already near the useful complexity boundary.

9 Discussion

The results and architecture suggest a useful reframing of the project. The central claim should not be that a CNN alone is a universally superior GKP decoder. A more defensible claim is that drift-adaptive GKP correction can be organized as a low-dimensional affine calibration problem with a deterministic fast path and replaceable slow calibration modules. Under this framing, the teacher-anchored CNN branch, the statistical calibration branch, and the filtering baselines are all instances of the same runtime question: how should recent syndrome statistics update (K, b) without disturbing the per-shot latency contract?

This framing clarifies why the method is different from several neighboring approaches. It is not a conventional FPGA surface-code decoder, because it does not operate on a binary detection-event graph. It is not an outer-code GKP soft-information decoder, because it updates the physical displacement rule before outer-code message passing or matching. It is not an end-to-end neural QEC decoder, because the neural component is not placed on the per-shot critical path. It is also not simply a calibration pre-processing routine, because the estimated state is committed through the same quantized, clipped, double-buffered runtime interface that the fast loop consumes.

The advantage should be interpreted at two levels. At the mainline project level, the strongest accepted result is still the locked four-scenario software-HIL ranking: the teacher-anchored hybrid branch beats the filtering baselines across all frozen scenarios. At the model-comparison level, the current evidence does not require claiming that the CNN is always the best estimator. A bounded statistical-calibration extension lane is instead useful because it shows that the same runtime contract can host a strong non-neural alternative; however, the accepted closure still keeps that lane supplement-side and no-promotion because the default and high-threshold settings remain persistently tied. The learned branch therefore remains valuable as a replaceable slow-loop module rather than as a uniquely validated final comparator.

The expected long-term advantage is therefore modularity under real-time constraints. If a future device requires a stronger statistical estimator, a learned temporal model, or a more conservative fallback policy, the slow calibration module can be replaced without changing the affine hardware boundary. Conversely, if future GKP experiments require tighter latency, the fast path can remain a fixed-point matrix-vector operation while the slow estimator is improved offline or on a host processor. This is the main systems-level reason to prefer the proposed contract over a monolithic decoder design: the architecture can trade estimator sophistication against update cadence without putting neural inference or a graph decoder on the per-shot critical path.

The boundary of the current evidence remains important. Software-HIL results support the adaptive-affine calibration story under controlled drift scenarios; isolated current-host `.tfllite` execution supports a narrow exported-model runtime claim; and read-only real-board gate/regeneration/provenance checks preserve a current-host `NO_GO` hardware boundary. These layers do not yet establish board-level latency, FPGA resource use, or closed-loop feedback timing. Those measurements are the natural next evidence layer if the work is to move from “hardware-compatible” to hardware-demonstrated.

10 Conclusion

This work frames drift-adaptive GKP correction as a two-timescale affine calibration problem. The fast loop remains a bounded fixed-point rule, $\Delta_t = K_t s_t + b_t$, while a slower loop updates the affine parameters from recent syndrome statistics. This formulation creates a practical middle ground between fixed decoders, which cannot track hardware drift, and full neural decoders, which can be difficult to place on a deterministic per-shot path.

The numerical results support two main conclusions. First, under the locked four-scenario software-HIL suite, adaptive affine decoding with a teacher-anchored residual branch improves over filtering baselines, while bounded ablations show that histogram-delta features carry useful information and that teacher-side inputs do not admit a simple necessity story. Second, a separately labeled statistical-calibration extension lane shows that simple non-neural estimators can also be competitive under the same runtime contract, but the accepted closure remains no-promotion and no unique clean threshold. Together, these results suggest that the most robust scientific story is not simply that a CNN is better than classical filters, but that low-dimensional histogram-driven calibration of an affine GKP fast path is a promising structure for drift-adaptive decoding.

The remaining technical gap is to connect this bounded software-HIL evidence to broader drift families, stronger oracle and wrapped-Gaussian baselines, measured runtime constraints, and eventually real hardware timing. Current deployment-facing material remains layered: isolated true runtime is verified only on the current host, while the real-board lane remains gate-only and NO_GO. Those additions would test whether the affine calibration contract remains useful when the benchmark moves from a controlled drift suite toward a deployment-grade decoder evaluation. If successful, the resulting contribution would be a practical bridge between analog GKP decoding theory and the low-latency FPGA-QEC systems literature: a physical-layer calibration module that preserves analog information, tracks drift, and still exposes a compact deterministic fast path.

References

- [1] D. Gottesman, A. Kitaev, and J. Preskill, “Encoding a qubit in an oscillator,” *Physical Review A*, 2001.
- [2] A. L. Grimsmo and S. Puri, “Quantum Error Correction with the Gottesman–Kitaev–Preskill Code,” *PRX Quantum* 2, 020101, 2021.
- [3] K. Fukui, A. Tomita, A. Okamoto, and K. Fujii, “High-threshold fault-tolerant quantum computation with analog quantum error correction,” *Physical Review X*, 2018.
- [4] K. Noh and C. Chamberland, “Fault-tolerant bosonic quantum error correction with the surface-GKP code,” *Physical Review A*, 2020.
- [5] K. Noh, C. Chamberland, and F. G. S. L. Brandao, “Low-overhead fault-tolerant quantum error correction with the surface-GKP code,” *PRX Quantum*, 2022.
- [6] N. Raveendran, N. Rengaswamy, F. Rozpedek, A. Raina, L. Jiang, and B. Vasic, “Finite-rate QLDPC-GKP coding scheme that surpasses the CSS Hamming bound,” *Quantum*, 2022.
- [7] L. Berent et al., “Analog information decoding of bosonic quantum LDPC codes,” *PRX Quantum*, 2024.
- [8] S. K. Borah, A. K. Pradhan, N. Raveendran, M. Pacenti, and B. Vasic, “Fault tolerant decoding of QLDPC-GKP codes with circuit level soft information,” preprint, 2025.

- [9] M.-A. Roy, T. Pousset, and B. Royer, “Decoding multimode Gottesman–Kitaev–Preskill codes with noisy auxiliary states,” preprint, 2025.
- [10] S. Spitz, B. Tarasinski, C. W. J. Beenakker, and T. E. O’Brien, “Adaptive weight estimator for quantum error correction,” *Advanced Quantum Technologies*, 2018.
- [11] T. Wagner, H. Kampermann, and D. Brūß, “Optimal noise estimation from syndrome statistics of quantum codes,” *Physical Review Research*, 2021.
- [12] H. Chen et al., “DGR: Tackling drifted and correlated noise in quantum error correction via decoding graph re-weighting,” preprint, 2023.
- [13] E. H. Chen et al., “Calibrated decoders for experimental quantum error correction,” *Physical Review Letters*, 2022.
- [14] V. Sivak, M. Newman, and P. Klimov, “Optimization of decoder priors for accurate quantum error correction,” *Physical Review Letters* 133, 150603, 2024.
- [15] C. Chamberland, J. Olle, M. Li, S. Thornton, and I. Baratta, “Fast and accurate AI-based pre-decoders for surface codes,” arXiv:2604.12841, 2026.
- [16] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, “Calibration-conditioned FiLM decoders for low-latency decoding of quantum error correction evaluated on IBM repetition-code experiments,” preprint, 2026.
- [17] Google Quantum AI and collaborators, “Learning high-accuracy error decoding for quantum processors,” preprint, 2023.
- [18] “LILLIPUT: A lightweight low-latency lookup-table based decoder for near-term quantum error correction,” *ASPLOS*, 2022.
- [19] “AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers,” *HPCA*, 2022.
- [20] “Scalable quantum error correction for surface codes using FPGA,” preprint, 2023.
- [21] “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching acceleration,” *ISCA*, 2023.
- [22] “A real-time, scalable, fast and resource-efficient decoder for a quantum computer,” *Nature Electronics*, 2025.
- [23] “FPGA-tailored algorithms for real-time decoding of quantum LDPC codes,” preprint, 2025.
- [24] L. Caune et al., “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” preprint, 2024.
- [25] X. Yang et al., “Real-time surface-code error correction using an FPGA-based neural-network decoder,” preprint, 2026.
- [26] M. Ziad et al., “Local clustering decoder: a fast and adaptive hardware decoder for the surface code,” preprint, 2024.
- [27] L. Maurer et al., “Real-time decoding of the gross code memory with FPGAs,” preprint, 2025.

- [28] V. Sivak et al., “Real-time quantum error correction beyond break-even,” *Nature*, 2023.
- [29] D. Lachance-Quirion et al., “Autonomous quantum error correction of Gottesman–Kitaev–Preskill states,” *Physical Review Letters*, 2024.

Metric	Advantage	Architectural reason	Evidence boundary
Logical-error proxy	Lower error under controlled drift than filtering baselines; statistical calibration gives an even stronger non-neural reference point.	Both learned and statistical branches update the same physical-layer affine surface, so adaptation acts directly on the displacement rule used by the GKP fast loop.	Supported in the four-scenario software-HIL suite; not a paper-grade expanded benchmark.
Per-shot latency	Low latency by design on the correction path.	The fast loop evaluates only a 2×2 affine map, bias addition, clipping, quantization, and parameter-bank selection; CNN or statistical estimation is kept off the per-shot critical path.	Architectural claim; board timing and worst-case latency remain a hardware measurement target.
FPGA resource and cost	Expected small fast-path resource footprint and lower verification burden than placing a neural or graph decoder in the real-time path.	The real-time datapath needs only fixed-point multiply-adds, saturation logic, and a compact parameter bank; the slow estimator can run less frequently on a host, embedded processor, or offline update service.	Resource use and energy/cost are not yet measured on a real device.
Engineering implementation	The control boundary is simple to integrate, test, and fail safely.	Quantized (K, b) , clipping limits, stale-parameter handling, saturation counters, and double-buffered stage-and-commit updates form explicit runtime contracts.	Software-HIL validates the interface shape; real-board execution is outside the current evidence layer.
Drift robustness	Adaptation can track mean, variance, and orientation changes in the effective GKP noise state.	Histogram windows and teacher/statistical summaries update the affine parameters over time instead of freezing one calibration point.	Supported for the current drift suite; broader drift families and hold-out protocols remain benchmark-expansion targets.
Modularity	The best slow-loop estimator can change without changing the fast path.	Teacher residual CNN, statistical calibration, temporal models, and gain-scheduled parameter banks all map to the same committed (K, b) interface.	This is the reason a strong statistical-calibration result strengthens the project-level contract rather than invalidating it.
Compatibility with larger QEC stacks	The method can serve as a GKP physical-layer calibration module before an outer stabilizer, surface-GKP, or LDPC-GKP decoder.	It improves the physical displacement correction and residual analog statistics instead of replacing the outer-code decoder decision rule.	System-level concatenated-code gains remain future-work evidence.

Table 1: Metric-level advantage summary. The strongest current evidence is logical-error reduction in software-HIL. Latency, resource, and cost advantages are architecture-level consequences of the affine fast path and should be measured on hardware before being written as deployment results.

Scenario	EKF	UKF	Const.- μ	RLS- b	Hybrid- b
<code>static_bias_theta</code>	0.838110	0.825370	0.836658	0.837577	0.810902
<code>linear_ramp</code>	0.819200	0.811201	0.816911	0.819373	0.787755
<code>step_sigma_theta</code>	0.822365	0.811548	0.819784	0.821493	0.788800
<code>periodic_drift</code>	0.832192	0.821558	0.829670	0.832334	0.806392

Table 2: Four-scenario affine benchmark. Entries are `final_ler_mean`; lower is better.

Mode	Avg. LER	Δ vs UKF	Δ vs Hybrid Full
ukf	0.817382	0.000000	+0.018837
hybrid_full	0.798545	-0.018837	0.000000
hybrid_no_hist_deltas	0.826723	+0.009341	+0.028178
hybrid_no_teacher_prediction	0.807251	-0.010131	+0.008706
hybrid_no_teacher_params	0.749621	-0.067761	-0.048924
hybrid_no_teacher_deltas	0.800329	-0.017053	+0.001784

Table 3: Feature and teacher ablation summary. Negative Δ means improvement over the reference.

Scenario	UKF	Hybrid- <i>b</i>	StatCalib
static_bias_theta	0.825370	0.810902	0.431708
linear_ramp	0.811201	0.787755	0.467083
step_sigma_theta	0.811548	0.788800	0.460016
periodic_drift	0.821558	0.806392	0.438751

Table 4: Bounded statistical-calibration extension-lane comparison under the same drift suite. Entries are `final_ler_mean`; lower is better. This table does not by itself promote statistical calibration into the mainline comparator set.

Variant	Mean LER	Mean Rank	Scenario Wins
statcalib_high_threshold	0.449241	1.75	1
statcalib_default	0.449254	1.25	3
statcalib_high_scale	0.456477	3.00	0
statcalib_low_clip	0.481484	4.50	0
statcalib_low_scale	0.485129	4.50	0

Table 5: Local sensitivity within the bounded statistical-calibration extension lane. Mean rank is computed within the five variants for each scenario; the near-tie between default and high-threshold prevents a unique promoted setting.

Scenario	Best variant	Best LER	Gap vs Hybrid- <i>b</i>
static_bias_theta	high_threshold	0.431684	0.379889
linear_ramp	default	0.467094	0.321178
step_sigma_theta	default	0.458834	0.328883
periodic_drift	default	0.439352	0.367232

Table 6: Best statistical-calibration variant per scenario within the bounded extension lane. Gap is the hybrid residual LER minus the best statistical calibration LER; this table does not remove the accepted no-promotion gate.